

# MLFF CUEQ-DEP1 Accelerator Runtime Freeze

Content-addressed dependency and CUDA-environment authority

mdstats project

2026-08-15

## Status

**Gate:** CUEQ-DEP1

**Implementation release:** mdstats 0.20.188a0

**Authority class:** runtime/dependency provenance; no scientific-model change

**Implementation status:** complete

**Accelerator qualification:** deferred to FINAL-GPU1

CUEQ-DEP1 freezes the exact accelerator environment that later CuEq phases are allowed to use. It does **not** establish numerical parity, training quality, or a generated CuEq default. Those decisions belong to CUEQ-PHASE1/CUEQ-PHASE2 and PERF-CERT1.

## 1. Required phase-1 execution contract

The first accelerator experiment is phase separated:

$$\text{source inference} = \text{e3nn}, \quad \text{training} = \text{CuEq}_{\text{pure}}.$$

For MACE 0.3.16, OpenEquivariance is therefore optional at CUEQ-DEP1. Its presence is recorded when installed, but it is not allowed to become an implicit phase-1 requirement.

The required CuEquivariance stack has three independently verified import/distribution layers:

1. `cuequivariance` - core representation and non-framework components;
2. `cuequivariance_torch` - PyTorch frontend;
3. `cuequivariance_ops_torch` - CUDA kernels supplied by a CUDA-major-specific distribution such as `cuequivariance-ops-torch-cu12` or `...-cu13`.

NVIDIA documents this split and the separate CUDA-ops installation explicitly [1]. MACE exposes CuEq acceleration through its `enable_cueq` training/inference interface [2].

## 2. Runtime authority

`CueqDep1RuntimeRecord.v1` binds:

- `CueqDep1Policy.v1`;
- the existing MACE/e3nn source-compatibility freeze;
- exact installed-distribution evidence for Torch, MACE, e3nn, CuEq core/Torch/ops, and optional OEQ;
- the imported module-root byte identity;
- CUDA-visible device inventory, compute capability, memory, driver/toolkit text, and cuDNN version;
- PyTorch deterministic-algorithm, cuDNN, TF32, and float32-matmul settings; and
- selected CUDA/CuEq environment variables.

For installed Python distributions the record stores SHA-256 identities of METADATA, RECORD, WHEEL, optional `direct_url.json`, and the imported module file. Python’s `importlib.metadata` defines the installed-distribution metadata/file view used here [3]. SHA-256 follows FIPS 180-4 [4].

The package-content predicate is

$$C_i = I_i \wedge H_i^{\text{METADATA}} \wedge H_i^{\text{RECORD}} \wedge H_i^{\text{module}},$$

where  $I_i$  denotes a successful import and each  $H$  term denotes a present content identity. A required component without this evidence is not considered frozen.

### 3. Gate predicate

Let

- $R_{\text{MACE}}$  be the locked MACE/e3nn runtime/source contract;
- $V$  be exact MACE/e3nn version agreement;
- $C$  be content-addressed identity for every required distribution;
- $A$  be CUDA plus CuEq core/Torch/ops capability.

Then

$$Q_{\text{CUEQ-DEP1}} = R_{\text{MACE}} \wedge V \wedge C \wedge A.$$

For phase 1, OEQ is excluded from  $C$  and  $A$ . If a later policy explicitly requires OEQ, it becomes a required component and the same fail-closed rule applies.

No failed term triggers e3nn substitution. A negative record is valid evidence that the accelerator gate is **not** qualified.

### 4. CUDA-major package discovery

The ops import is stable (`cuequivalence_ops_torch`) while the distribution is CUDA-major specific. Discovery order is:

```
cuequivalence-ops-torch-cu13
cuequivalence-ops-torch-cu12
cuequivalence-ops-torch-cu11
cuequivalence-ops-torch
```

This is capability discovery only. CUEQ-DEP1 freezes whichever installed distribution actually supplied the import; it does not infer compatibility from the package name alone.

### 5. Final-release capture

The source package contains two entry points:

```
python tools/capture_mlff_cueq_dep1_runtime.py \
  --output CUEQ_DEP1_RUNTIME_FREEZE.json
```

and the consolidated release preflight:

```
python tools/run_mlff_final_gpu_qualification.py \
  --mh1-model /path/to/mace-mh-1.model \
  --mpa0-model /path/to/mace-mpa-0-medium.model \
  --output FINAL_GPU1_PREFLIGHT.json
```

The latter embeds the complete CUEQ-DEP1 runtime record and treats CUEQ-DEP1 as a blocker for the later CuEq phases.

The development host intentionally produces a negative record because it has CPU-only PyTorch and no CuEq packages. That record proves fail-closed behavior but does not qualify the gate.

## 6. Acceptance

Implementation qualification requires:

- schema round-trip and digest-tamper rejection;
- explicit CuEq ops-layer discovery including CUDA 13 and CUDA 12 distributions;
- OEQ optionality for phase 1 and fail-closed promotion when a policy requires it;
- negative CUDA evidence without backend fallback;
- synchronization of the standalone capture tool and **FINAL-GPU1** preflight; and
- unchanged e3nn/MACE reference behavior on the supplied foundation models.

Final CUEQ-DEP1 qualification additionally requires one **passed=true** runtime record captured on the final release-matched accelerator machine. That evidence remains pending by design until **FINAL-GPU1**.

## References

1. NVIDIA, *cuEquivariance Documentation - Installation*, <https://docs.nvidia.com/cuda/cuequivariance/>.
2. MACE documentation, *CUDA Acceleration with cuEquivariance Library*, [https://mace-docs.readthedocs.io/en/latest/guide/cuda\\_acceleration.html](https://mace-docs.readthedocs.io/en/latest/guide/cuda_acceleration.html).
3. Python documentation, *importlib.metadata - Accessing package metadata*, <https://docs.python.org/3/library/importlib.metadata.html>.
4. NIST, *FIPS 180-4: Secure Hash Standard*, <https://doi.org/10.6028/NIST.FIPS.180-4>.